

```

1 public class Main {
2     public static void main(String[] args) {
3         Room room = new Room(new char[][] {
4             {'#', '#', '#', '#', '#', '#', '#',
5             '#', '#', '#', '#', '#'},
6             {'#', ' ', ' ', ' ', ' ', ' ', ' ',
7             ' ', ' ', ' ', ' ', '#'},
8             {'#', ' ', ' ', ' ', ' ', ' ', ' ',
9             ' ', ' ', ' ', ' ', '#'},
10            {'#', ' ', ' ', ' ', ' ', ' ', ' ',
11            ' ', ' ', ' ', ' ', '#'},
12            {'#', ' ', ' ', ' ', ' ', ' ', ' ',
13            ' ', ' ', ' ', ' ', '#'},
14            {'#', '#', '#', '#', '#', '#', '#',
15            '#', '#', '#', '#', '#'}));
16        System.out.println("Initial room layout:");
17        room.print();
18        Position start = new Position(6, 7);
19        Position target = new Position(1, 1);
20        System.out.println("\nDistance matrix for
target (1,1):");
21        room.printDistanceMatrix(target);
22        System.out.println("\nShortest path from
start (6,7) to target (1,1):");
23        room.printShortestPath(start, target);
24    }
25 }

```

```
1 import java.util.ArrayList;
2 import java.util.List;
3
4 public class Room {
5
6     private char[][] layout;
7
8     public Room(char[][] layout){
9         this.layout = layout;
10    }
11
12    private boolean isAccessible(Position position
13    ){
14        int x = position.getX();
15        int y = position.getY();
16
17        return x >= 0
18            && y >= 0
19            && layout[x][y] != '#';
20    }
21
22    private int[][] getDistanceMatrix(Position
23    position) {
24        int[][] matrix = new int [layout.length][
25        layout[0].length];
26        for (int i = 0; i < matrix.length; i++) {
27            for (int j = 0; j < matrix[0].length; j
28            ++){
29                matrix[i][j] = -1;
30            }
31        }
32        matrix[position.getX()][position.getY()] =
33        0;
34        Position actPos = position;
35        List<Position> queue = new ArrayList<>();
36        boolean over = false;
37        while(!over) {
38            Position[] neighbors = actPos.
39            getNeighbors();
40            for (int i = 0; i < neighbors.length; i
41            ++){
```

```

35         if (isAccessible(neighbors[i]) &&
matrix[neighbors[i].getX()][neighbors[i].getY
()] == -1) {
36             matrix[neighbors[i].getX()][
neighbors[i].getY()] = matrix[actPos.getX()][actPos
.getY()] + 1;
37             queue.add(neighbors[i]);
38         }
39     }
40     if (!queue.isEmpty()) {
41         actPos = queue.remove(0);
42     } else {
43         over = true;
44     }
45 }
46 return matrix;
47 }
48
49 public void print(){
50     for (int i = 0; i < layout.length; i++) {
51         for (int j = 0; j < layout[i].length; j
++) {
52             System.out.printf("%3c", layout[i][
j]);
53         }
54         System.out.println();
55     }
56 }
57
58 public void printDistanceMatrix(Position
position){
59     int[][] distanceMatrix = getDistanceMatrix(
position);
60     for (int i = 0; i < distanceMatrix.length;
i++) {
61         for (int j = 0; j < distanceMatrix[i].
length; j++) {
62             System.out.printf("%3d",
distanceMatrix[i][j]);
63         }
64         System.out.println();

```

```
65     }
66 }
67
68 public void printShortestPath(Position start,
    Position target){
69     int[][] matrix = getDistanceMatrix(target
    );
70     Position[] shortestWay = new Position[
    matrix[start.getX()][start.getY()]];
71
72     Position actPos = start;
73     for (int i = 0; i < matrix[start.getX()][
    start.getY()]; i++) {
74         Position[] neighbors = actPos.
    getNeighbors();
75         for (int j = 0; j < neighbors.length;
    j++) {
76             if (isAccessible(neighbors[j]) &&
    matrix[start.getX()][start.getY()] > matrix[
    neighbors[j].getX()][neighbors[j].getY()]) {
77                 shortestWay[i] = actPos;
78                 actPos = neighbors[j];
79                 break;
80             }
81         }
82     }
83
84     for (int i = 0; i < shortestWay.length; i
    ++){
85         System.out.print(shortestWay[i] +
    " -> ");
86     }
87     System.out.print(target);
88 }
89
90 }
91
```

```
1 import java.util.Objects;
2
3 public class Position {
4
5     private int x;
6     private int y;
7
8     public Position(int x, int y){
9         this.x = x;
10        this.y = y;
11    }
12
13    public int getY() {
14        return y;
15    }
16
17    public int getX() {
18        return x;
19    }
20
21    public void setX(int x) {
22        this.x = x;
23    }
24
25    public void setY(int y) {
26        this.y = y;
27    }
28
29    public Position[] getNeighbors() {
30        Position[] neighbors = new Position[4];
31
32        neighbors[0] = new Position(x, y + 1);
33        neighbors[1] = new Position(x + 1, y);
34        neighbors[2] = new Position(x, y - 1);
35        neighbors[3] = new Position(x - 1, y);
36
37        return neighbors;
38    }
39
40    public String toString() {
41        return "(" + x + ", " + y + ")";
```

```
42     }
43
44     @Override
45     public boolean equals(Object o) {
46         if (o == null || getClass() != o.getClass
47         ()) return false;
48         Position position = (Position) o;
49         return x == position.x && y == position.y;
50     }
51
52     @Override
53     public int hashCode() {
54         return Objects.hash(x, y);
55     }
56 }
```